

# EE5203 L02 Study Sheet

Values, Decisions, Loops, and Arrays - Basri Erdogan

**Essential question:** How does one C loop turn eight raw sensor values into a validated average, a count, and a status decision — and how do you prove each intermediate value in the debugger?

## What you should be able to do

- Trace assignment and arithmetic statements in execution order.
- Predict the result of integer division and remainder.
- Distinguish = (store) from == (compare).
- Write and trace if, else if, else decision chains.
- Trace while and for loops, including first and last counter values.
- Index an array safely and explain why `samples[8]` is invalid for `[8]`.
- Use the accumulator pattern to compute a sum and average.

## Core statements

| Statement                             | Question it answers                                        |
|---------------------------------------|------------------------------------------------------------|
| <code>x = 5;</code>                   | Nothing — it <b>stores</b> 5 in <code>x</code>             |
| <code>x == 5</code>                   | Does <code>x</code> currently contain 5?                   |
| <code>if (c) { ... }</code>           | Run the block once <b>if</b> <code>c</code> is true        |
| <code>while (c) { ... }</code>        | Repeat <b>while</b> <code>c</code> stays true (test first) |
| <code>for (init; test; update)</code> | Repeat with a counter over a known range                   |
| <code>while (1) { ... }</code>        | Firmware main loop — repeat forever                        |

## Operators

| Group      | Operators                                | Watch out                                                            |
|------------|------------------------------------------|----------------------------------------------------------------------|
| Arithmetic | <code>+ - * / %</code>                   | <code>10 / 3</code> is 3 (integer); <code>10 % 3</code> is 1         |
| Shorthand  | <code>x++, x += 5</code>                 | Same as <code>x = x + 1</code> , <code>x = x + 5</code>              |
| Comparison | <code>== != &lt; &lt;= &gt; &gt;=</code> | Comparing changes <b>neither</b> operand                             |
| Logical    | <code>&amp;&amp;    !</code>             | <code>&amp;&amp;</code> both true; <code>  </code> at least one true |

## The for loop, traced

```
for (int i = 0; i < 4; i++) { /* body */ }
```

| Repetition | i before body | i < 4? |
|------------|---------------|--------|
| 1          | 0             | yes    |
| 4          | 3             | yes    |
| stop       | 4             | no     |

**START** creates `i = 0`, **TEST** runs before every repetition, **UPDATE** runs after every body. The body runs four times — `i` never reaches 4 inside the body.

## Arrays

```
#include <stdint.h>

uint16_t samples[8] = { 850, 1250, 4095, 4200,
                        2050, 900, 3100, 2800 };
```

- `uint16_t`: unsigned, 16-bit, fixed-width integer — no negative values.
- Valid indices are 0 through 7. `samples[8]` is **outside** the array.
- Loop condition is `i < 8`, never `i <= 8`.

## The accumulator pattern

```
int sum = 0; // must start at zero

for (int i = 0; i < 8; i++) {
    if (samples[i] > 4095) { // clamp invalid input
        samples[i] = 4095;
    }
    sum += samples[i]; // accumulate
    if (samples[i] > 2500) {
        high_count++; // count while visiting
    }
}

int average = sum / 8; // integer division
```

One loop can clamp, sum, and count in a single pass. Every result variable needs a correct starting value **before** the loop.

## Decision chains

```
if (average < 1500)    status_code = -1; // LOW
else if (average > 3000) status_code = 1;  // HIGH
else                  status_code = 0;  // OK
```

Conditions are tested top to bottom; exactly one branch runs; testing stops at the first true condition.

## Common mistakes

- Writing `=` where `==` is meant — the condition becomes an assignment.
- Forgetting to initialize an accumulator (`sum` starts with garbage).
- `i <= 8` on an eight-element array — reads/writes outside the array.
- Forgetting the update in a `while` loop — the loop never ends.
- Expecting `19140 / 8` to have a fraction — integer division discards it.
- Trusting a clean build as proof the computed values are correct.

## Mastery self-check

- ☐ I can trace `x = x + 3; y *= 5; x++;` and give both final values.
- ☐ I can explain why `if (status = 1)` is a bug, not a comparison.
- ☐ I can state how many times `for (int i = 0; i < 8; i++)` runs its body.
- ☐ I can list the valid indices of `uint16_t samples[8]`.
- ☐ I can explain why `sum` must start at zero.
- ☐ I can predict `sum / 8` when `sum` is 19140.
- ☐ I can place a breakpoint inside a loop and read `i` and `samples[i]`.

## Five review questions

1. What are the final values of `a` and `b` after `int a = 6, b = 4; a = a - b; b++; a *= 2;`?
2. `int x = 7;` — what is the value of the expression `x % 3`, and what is `x` afterwards?
3. A `while` loop's body never executes. What must be true of its condition, and when was it tested?
4. In one pass over an array, which of these need a starting value before the loop: the loop counter, an accumulator, a count of high samples?
5. The lab program clamps 4200 to 4095 **before** adding it to `sum`. What would the average be wrong by if it clamped after summing?

See the L02 worksheet for extended practice. Worked solutions are released after the practice window.